

Azure DevOps Terraform YAML Pipeline

This document explains each section of the Azure DevOps YAML pipeline step by step. The pipeline automates Terraform deployments to Azure by installing Terraform, creating backend resources, initializing Terraform, generating a plan, and applying infrastructure changes.

This YAML file is an **Azure DevOps Pipeline** that automates Terraform deployment to Azure. Its main purpose is:

1. Install Terraform.
2. Login to Azure.
3. Create Terraform backend resources if they don't exist.
4. Initialize Terraform with remote state storage.
5. Generate an execution plan.
6. Apply the infrastructure changes automatically.

1. Trigger

trigger:

- main

This section automatically starts the pipeline whenever code is pushed to the main branch.

2. Agent Pool

pool:

vmImage: 'ubuntu-latest'

Azure DevOps creates a temporary Ubuntu virtual machine to execute the pipeline.

3. Variables

Variables store reusable values such as the Azure service connection, backend resource group, storage account, blob container, Terraform state file, and Azure region.

4. Install Terraform

TerraformInstaller@1 installs Terraform version 1.12.2 on the build agent.

5. Azure Login and Backend Creation

The AzureCLI task authenticates using the Azure service connection and performs the following:

- Checks if the Resource Group exists.
- Creates it if missing.
- Checks if the Storage Account exists.
- Creates it if missing.
- Retrieves the storage account access key.
- Creates the blob container if it does not already exist.

This design is idempotent, meaning the pipeline can be run multiple times safely.

6. Terraform Init

TerraformTaskV4 executes 'terraform init'.

It configures the Azure remote backend using:

- Resource Group
- Storage Account
- Blob Container
- State File

The '-reconfigure' option forces Terraform to refresh backend settings.

7. Terraform Plan

Terraform executes 'terraform plan'.

It compares the current Azure infrastructure with the desired configuration in the Terraform files and shows proposed changes without making modifications.

8. Terraform Apply

Terraform executes 'terraform apply'.

The '-auto-approve' option skips manual confirmation.

The '-lock=false' option disables state locking, though enabling locking is recommended for production.

9. Pipeline Flow

Push to main branch

↓

Pipeline starts

↓

Create Ubuntu agent

↓

Install Terraform

↓
Login to Azure
↓
Create backend resources if needed
↓
Terraform Init
↓
Terraform Plan
↓
Terraform Apply
↓
Infrastructure deployed

10. Advantages

- Automatic execution.
- Secure Azure authentication.
- Remote Terraform state storage.
- Safe and repeatable backend creation.
- Separate planning and deployment stages.
- Fully automated infrastructure provisioning.

Conclusion

This Azure DevOps YAML pipeline provides a complete CI/CD workflow for Terraform on Azure. It automatically provisions the Terraform backend, initializes Terraform, generates an execution plan, and deploys Azure infrastructure while following infrastructure-as-code best practices.